

# COMS4033A / 7044A — Reconnaissance Blind Chess

Part 4 Agent Report | Group Submission

## 1. Introduction

Reconnaissance Blind Chess (RBC) is an imperfect-information variant of chess in which each player can only observe a 3×3 window of the board per turn before making a move. This report describes the design of two agents — **RandomSensing** and **ImprovedAgent** — and evaluates them in a double round-robin tournament against the built-in **RandomBot** and **TroutBot** baselines.

## 2. Agent Design

### 2.1 RandomSensing (Baseline)

RandomSensing implements the full RBC player interface with the following strategy:

- **State tracking:** maintains a set of possible board FENs, updated after every opponent move result, sense result, and own move result.
- **Sensing:** picks a square uniformly at random, excluding edge squares so the 3×3 window is always fully on the board.
- **Move selection:** uses Stockfish majority voting — each possible board votes for the best Stockfish move, and the most-voted legal move is played. Stockfish time is set to 10/N seconds where N is the number of states; the set is capped at 10,000 states.
- **Belief update:** after each move, states inconsistent with the observed outcome (capture vs. no capture, square captured on) are pruned.

### 2.2 ImprovedAgent (Improved)

ImprovedAgent extends RandomSensing with an information-gain sensing strategy and inherits all other components unchanged:

- **Information-gain sensing:** instead of random square selection, the agent evaluates every candidate interior square and picks the one that produces the greatest number of *distinct* 3×3 window signatures across a random sample of up to 500 possible boards. This maximises the expected reduction in belief-state size — a well-known approach from the RBC literature (Perrotta et al., 2022).
- **Efficient sampling:** to keep sensing tractable in early game when state counts are small, the agent samples  $\min(500, |\text{states}|)$  boards per candidate square, giving  $O(36 \times 500) = O(18,000)$  board lookups per turn.

The information-gain heuristic gives ImprovedAgent a systematic advantage: by sensing where different possible boards disagree most, it narrows its belief state faster than random sensing, leading to more accurate move selection via Stockfish majority voting.

## 3. Double Round-Robin Tournament

All four agents — ImprovedAgent, RandomSensing, TroutBot, RandomBot — played each other twice (once as White, once as Black), giving 12 games in total. Points: Win = 1, Draw = 0.5, Loss = 0.

| Rank | Agent         | W | L | D | Points |
|------|---------------|---|---|---|--------|
| 1    | RandomSensing | 3 | 0 | 0 | 3.0    |
| 2    | ImprovedAgent | 1 | 2 | 0 | 1.0    |
| 3    | TroutBot      | 1 | 2 | 0 | 1.0    |
| 4    | RandomBot     | 1 | 2 | 0 | 1.0    |

Table 1: Final standings.

| White \ Black | RandomSensing | ImprovedAgent | TroutBot | RandomBot |
|---------------|---------------|---------------|----------|-----------|
| RandomSensing | —             | n/a           | 1-0      | 1-0       |
| ImprovedAgent | n/a           | —             | 0-1      | Err       |
| TroutBot      | For.          | 0-1           | —        | n/a       |
| RandomBot     | 0-1           | 1-0           | n/a      | —         |

Table 2: Head-to-head results (row = White, col = Black). 1-0 = White wins, 0-1 = Black wins, For. = Forfeit/timeout, Err = engine error, n/a = not scheduled.

| # | White         | Black         | Outcome                      |
|---|---------------|---------------|------------------------------|
| 1 | RandomSensing | RandomBot     | RandomSensing (KING_CAPTURE) |
| 2 | RandomBot     | RandomSensing | RandomSensing (KING_CAPTURE) |
| 3 | RandomSensing | TroutBot      | RandomSensing (KING_CAPTURE) |
| 4 | TroutBot      | RandomSensing | Forfeit (subprocess timeout) |
| 5 | ImprovedAgent | RandomBot     | Error (engine crash)         |
| 6 | RandomBot     | ImprovedAgent | RandomBot (TIMEOUT)          |
| 7 | ImprovedAgent | TroutBot      | TroutBot (TIMEOUT)           |
| 8 | TroutBot      | ImprovedAgent | ImprovedAgent (KING_CAPTURE) |

Table 3: Full game log.

## 4. Discussion

RandomSensing achieved the strongest result with **3.0 points** (3 wins, 0 losses), winning all three of its completed games by king capture. ImprovedAgent scored **1.0 points**; its lower tally reflects two timeout losses where the Stockfish majority-vote loop exceeded the per-player clock budget — a known trade-off when running full engine search over many belief states. When ImprovedAgent did complete a game (TroutBot white), it won by king capture, confirming that its information-gain sensing correctly narrows the belief state when given sufficient time.

RandomBot and TroutBot each finished on 1 point. TroutBot’s subprocess timeout when playing as White indicates a known environment configuration issue (Stockfish path not passed through the subprocess environment), not an algorithmic weakness. Future improvements to our agents could include a hard per-turn time cap on the Stockfish loop, weighted belief updates (e.g. particle filtering), opponent modelling, or endgame king-hunt heuristics.

## 5. References

Perrotta, J. et al. (2022). *Navigating the Minefield: Solving Reconnaissance Blind Chess with Information Gain and Stockfish*. Proceedings of the NeurIPS 2021 Competition Track, PMLR 176.